

PRÁCTICA PROFESIONAL SUPERVISADA INFORME FINAL

Alumno

Nombre: Gabriel Agustín Paladino

Numero de alumno: 02490/8

Teléfono: 2804881432

E-mail: gabrielpaladino24@alu.ing.unlp.edu.ar

Practica

Fecha de inicio: 19/8/2024

Fecha de fin: 26/03/2025

Horarios de concurrencia: 9 a 13hs

Profesor coordinador

Nombre: Thomas, Pablo

Teléfono y horario para ser contactado: 221-4227707, de 8 a 12 o de 16 a 20 hs

E-mail: pthomas@lidi.info.unlp.edu.ar

Tutor de la entidad

Nombre: Fava, Laura Andrea

Teléfono y horario para ser contactado: 221-4223528, de 8 a 17 hs

E-mail: lfava@info.unlp.edu.ar

Datos de la institución

Nombre: Laboratorio de Investigación en Nuevas Tecnologías Informáticas.

Dirección y teléfono: Calle 50 y 120, 2do. Piso, 422 3528

Página WEB: <http://linti.unlp.edu.ar/>

Sector productivo: Investigación y desarrollo.

Sector donde se desempeñará el practicante: Desarrollo de Software

Resumen

El plan de trabajo de la PPS se orientó a la elección del hardware y la creación del software para un dispositivo capaz recolectar datos sobre la posición y los movimientos de animales con el fin de poder rastrearlos y alertar en caso de anomalías. Esta PPS se enmarca en el Proyecto de Investigación y Desarrollo del LINTI, denominado “De la Sociedad del Conocimiento a la Sociedad 5.0: un abordaje tecnológico y ético en nuestra región”. Este informe describe el sistema informático obtenido, cuyo desarrollo requirió la aplicación de conocimientos en desarrollo de software y electrónica, y se hace énfasis en el uso de los dispositivos en modos de bajo consumo.

Contenido

Introducción	3
Actores	3
Sobre la entidad	3
Objetivo del proyecto	5
Etapas de desarrollo	6
Selección de tecnologías a utilizar	6
Diseño e implementación	6
Localización mediante GPS	6
Detección de anomalías mediante acelerómetro	7
Tipos de mensajes y configuración	8
Desarrollo de firmware	9
Red LoRa con Influx y Grafana	9
Pruebas y ensayos	9
Consumo eléctrico	9
Tiempo que toma en obtener señal	9
Resultados	10
Conclusiones	10

Introducción

Este informe trata el trabajo realizado por el alumno de Ingeniería en Computación de la Facultad Virtual de Informática-Ingeniería Gabriel Agustín Paladino, número de legajo 02490/8 a modo de práctica profesional supervisada a lo largo de los meses de Agosto de 2024 a marzo de 2025.

El trabajo consistió en la creación de las bases del proyecto “IoT en Agro” que se enmarca en el Proyecto de Investigación y Desarrollo del LINTI, denominado De la Sociedad del Conocimiento a la Sociedad 5.0: un abordaje tecnológico y ético en nuestra región.

Actores

Autor: Gabriel Agustín Paladino

Profesor coordinador: Pablo Thomas

Entidad: Laboratorio de Investigaciones en Nuevas Tecnologías Informáticas (LINTI)

Tutor de la Entidad: Laura Andrea Fava

Sobre la entidad

El Laboratorio de Investigación en Nuevas Tecnologías Informáticas (LINTI) fue creado en 1995 con el objetivo de conformar un espacio de investigación y formación de un grupo de docentes investigadores interesados en desarrollar trabajos innovadores en TICs con foco en educación, inclusión, gobierno y salud.

Desde su creación, el LINTI se centró en desarrollar áreas de investigación en tecnologías emergentes en torno a las redes de comunicaciones, seguridad y desarrollo de aplicaciones.

En la actualidad, continúa haciendo hincapié en temáticas innovadoras incursionando tópicos sobre la Internet del futuro como ciudades digitales inclusivas y sustentables, IoT y ciberseguridad.

Se puede ver en la figura 1 un organigrama de personal donde se muestra las personas que constituyen el LINTI y la jerarquía que las relaciona. También se puede ver en la figura 2 un organigrama de las actividades que se llevan a cabo en el laboratorio. Este proyecto se da en la línea de investigación <<Innovación>>

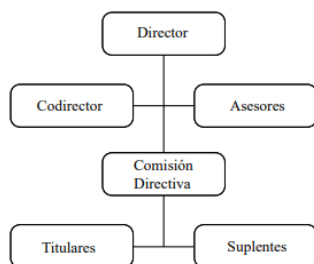


Figura 1: Organigrama de personal del LINTI

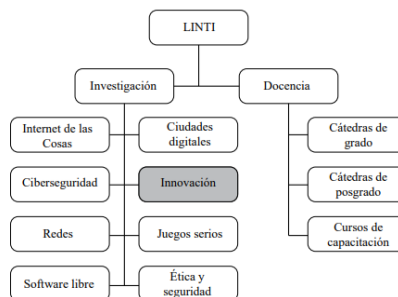


Figura 2: Organigrama de actividades del LINTI

Objetivo del proyecto

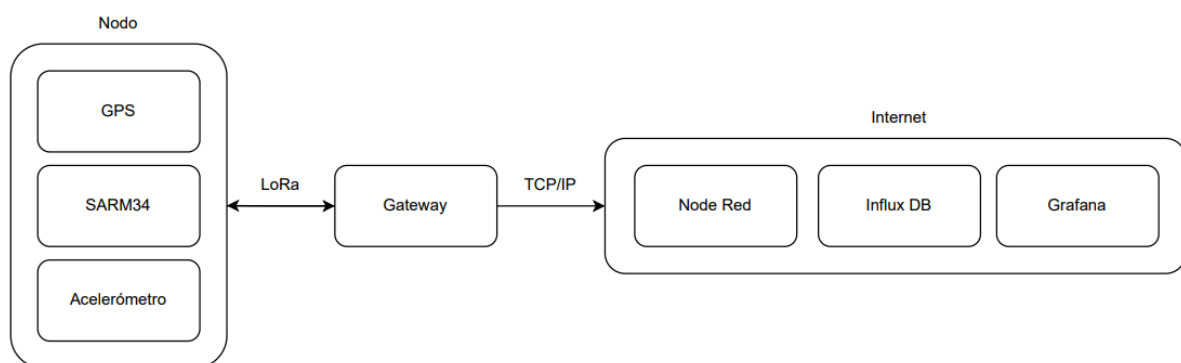
El proyecto se da en el contexto de la tecnología aplicada a la ganadería, específicamente en el tracking de animales permitiendo a los veterinarios obtener datos relevantes que le permitan detectar anomalías y enfermedades en los animales de manera temprana. El objetivo del proyecto se centra en crear el hardware y el firmware base que permita enviar la posición de los animales a un nodo Gateway cada un periodo de tiempo programable o en ciertas ocasiones denominadas anomalías.

El sistema está formado por:

- Nodos o dispositivos en forma de collar que funcionarán de manera inalámbrica mediante baterías e irán montados sobre cada animal a monitorear.
- El Gateway que se encarga de gestionar y almacenar los datos provenientes de los nodos como así configurar los nodos de manera remota.

El desarrollo del sistema se dividió en las siguientes partes:

- Localización del dispositivo. Se trata de obtener la latitud y longitud en el que se encuentra el dispositivo mediante la conexión y programación del GPS.
- Detectar anomalías en los movimientos del animal. Utilización de acelerómetro para obtener datos de los movimientos y así crear rutinas que permitan evaluar caídas o movimientos demasiados rápidos para el animal y permitan enviar una alerta.
- Creación del sistema de mensajes. Debemos dar formato y determinar que puertos de LoRa se utilizaran para los distintos tipos de mensajes.
- Conexión a la red y manejo de datos. Se trata de conectar los nodos mediante LoRa al Gateway y a la red ya existente en la facultad para almacenar los datos en “InfluxDB” y diseñar una interfaz gráfica que muestre datos relevantes con “Grafana”.

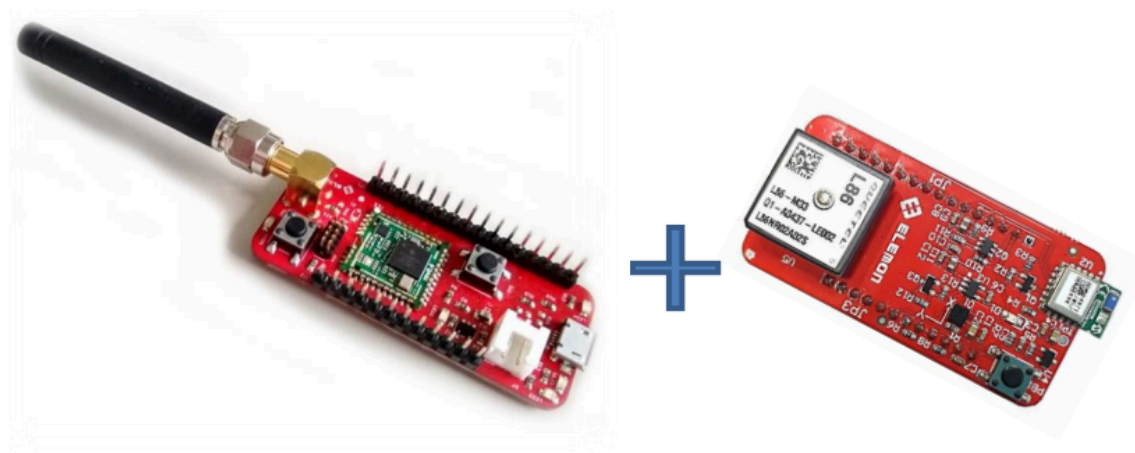


Etapas de desarrollo

El proyecto se puso en marcha el 20 de Agosto de 2024. Durante el desarrollo del mismo se escogió la electrónica y se realizó el firmware para que cumpla con las funciones básicas previamente establecidas.

Selección de tecnologías a utilizar

Se optó por utilizar placas de desarrollo EESARM35 de Elemon debido a que incluyen el micro SAM R35 que ya trae incorporado LoRa y está pensado para aplicaciones de bajo consumo, haciéndolo ideal para nuestro proyecto. Sumado a eso ofrecen un shield que va montado sobre dicha placa agregando varios periféricos más, entre ellos un acelerómetro y un GPS, obteniendo de esta manera todo lo necesario para llevar a cabo el proyecto en un tamaño muy compacto y evitando desarrollar una PCB.



**Placa Carrier SAMR34 / 35
Basado en Módulo LoRa.
Modulo + Cargador + Bridge UART-USB**

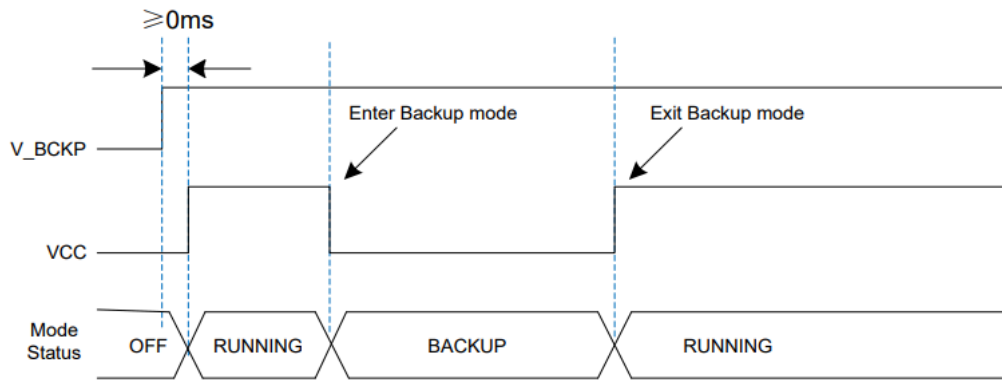
**Shield para placa Carrier.
BLE + GPS + Acelerómetro**

Diseño e implementación

El autor de este informe tuvo participación predominante en la investigación de la electrónica a utilizar y el desarrollo del firmware. Por tratarse de un informe acerca de las actividades realizadas por el autor, estos son los temas que se cubrirán.

Localización mediante GPS

El shield dispone del GPS Quectel L80, el cual se comunica mediante el protocolo i2c con el microcontrolador. El mismo cuenta con dos pines de alimentación, VCC y V_BCKP, los cuáles nos permiten elegir entre los modos de funcionamiento disponibles. El modo más relevante para este proyecto es el modo “backup”, ya que solo tiene un consumo de 7uA y nos permite obtener señal de GPS lo más rápido posible. Esto es gracias a que mantiene activa una memoria de respaldo que almacena información como la hora del GPS y la última posición obtenida. El mismo se activa dejando alimentado únicamente el pin V_BCKP como se muestra en la siguiente figura.



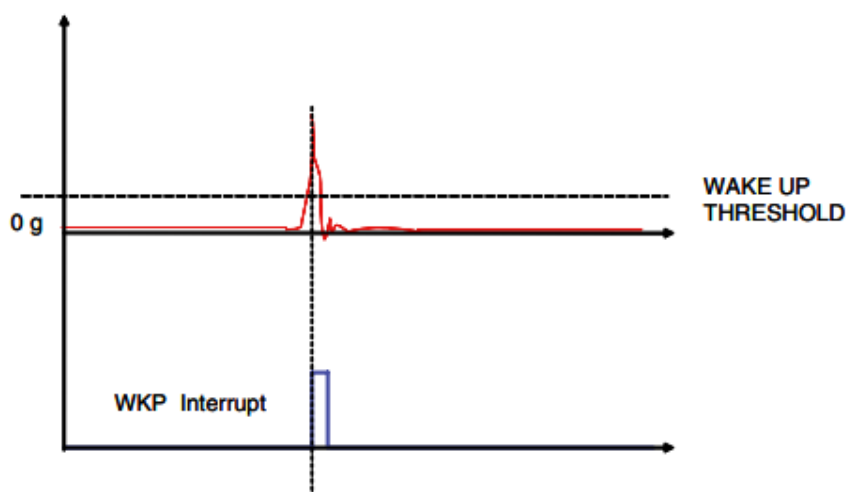
Cabe considerar que la placa de desarrollo está diseñada para que el pin V_BCKP siempre este alimentado y nos permite controlar VCC mediante el pin digital PA_18 para poder entrar en modo "backup" mediante código.

El código para utilizar el GPS ya venía incluido como ejemplo en el firmware de la placa, pero el mismo se readapto para que detecte sentencias vacías (GPS sin señal) y encienda un flag una vez que tome señal y obtenga los datos de su posición.

Detección de anomalías mediante acelerómetro

La placa cuenta con el chip LIS3DH, el cual es un acelerómetro lineal de tres ejes de alto rendimiento y consumo ultra bajo. Para este proyecto el mismo se configuro para que funcione a 50Hz en modo "low power" dando buenos resultados con un nivel de consumo muy bajo de aproximadamente 6uA.

El dispositivo también presenta funciones inteligentes integradas. Una de ellas es la posibilidad de generar una interrupción si uno de los ejes configurados supera cierto umbral de aceleración, como se muestra en la siguiente imagen.



Esta función fue utilizada para detectar aceleraciones que sean demasiado altas para los movimientos normales de un animal. El umbral que dispara la interrupción es configurable, por lo que se creó una función que nos permite escoger entre 5 niveles distintos de intensidad.

Dicha intensidad puede ser configurable mediante LoRa. Lo más interesante de todo esto es que no hace falta que el acelerómetro este en comunicación con el micro permitiéndole a este estar en modo sleep. Obteniendo de esta manera una alarma que detecte anomalías con un consumo ultra bajo y sin tener que realizar ninguna rutina más que la de configuración.

Tipos de mensajes y configuración

Para el envío y recepción de mensajes se debió dar formato y definir los tipos, como así también mediante qué puerto de LoRa se iban a transmitir. La siguiente tabla resume como se resolvió este tema.

Envío de datos		
Tipo	Datos	Puerto
Periódico	Latitud-Longitud-Batería	1
Alerta	Latitud-Longitud-Tipo	2
ACK Configuración	Horas-Minutos-Intensidad	3

Recepción de datos		
Tipo	Datos	Puerto
Configuración	Horas-Minutos-Intensidad	3

A continuación, se dan más detalles sobre los tipos mensajes, como el formato y los tipos de datos transmitidos.

El prototipo está pensado para que cada cierto periodo envíe mediante el puerto 1 de LoRa su ubicación y el estado de la batería con el siguiente formato: “Latitud-Longitud-Batería”, siendo latitud y longitud datos de tipo float que permiten localizar el dispositivo. Batería es un numero de 1 a 3 que indica el nivel de batería como se muestra en la siguiente imagen.

Nivel de batería		
Mensaje	Estado	Voltaje
1	Bajo	<3.5V
2	Medio	>3.5V y <3.9V
3	Alto	>3.9V

En caso de disparar algún tipo de alarma el formato será el mismo que el periódico, pero reemplazando el parámetro de batería por un número que indica que tipo de anomalía ocurrió y esta vez será enviado por el puerto 2 de LoRa. Para este caso el prototipo solo cuenta con la detección de caída, siendo este el tipo 1.

El prototipo también es capaz de recibir mensajes desde el Gateway, pero solo después de haber enviado algún mensaje ya que hacemos uso de LoRa clase A. El único tipo de mensaje que es capaz de recibir es de configuración para el periodo de envío de mensaje y para la

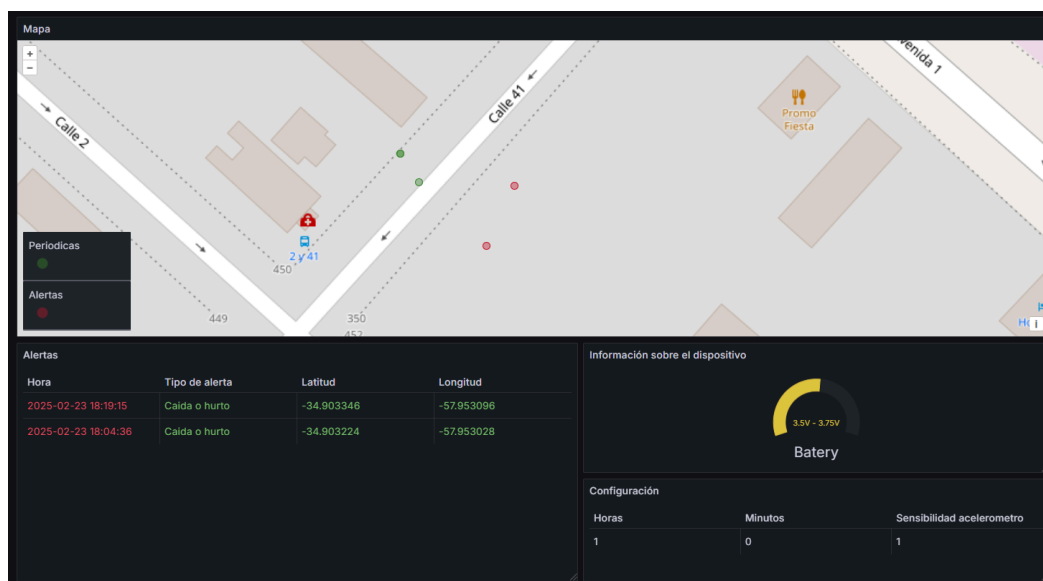
intensidad en la que dispare la alarma el acelerómetro. Dicho mensaje solo se recibe por el puerto 3 y debe cumplir con el siguiente formato: HHMMI. Donde H indica las horas y M los minutos del periodo en que se enviara la posición. Por último, I indica la sensibilidad de la alarma. La siguiente tabla muestra los requisitos que deben cumplir dichos parámetros.

Configuración		
Parámetro	Mínimo	Máximo
HH (horas)	0	24
MM (minutos)	1	59
I (sensibilidad)	1	5

Para asegurarnos de que las configuraciones se establecieron, el dispositivo envía un mensaje por el mismo puerto replicando los datos que enviamos en caso de éxito y en caso de que haya error en algún parámetro envía los parámetros que tenía configurados previamente y que prevalecerán en el dispositivo.

Dashboard

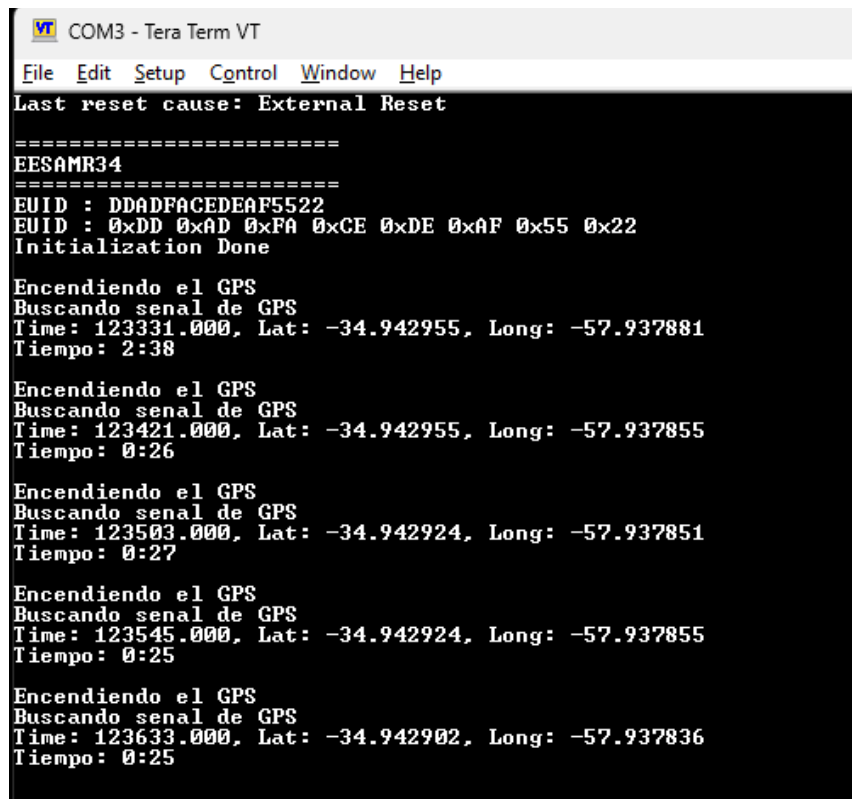
Para poder ver la información recibida y la del dispositivo de manera sencilla se creó un dashboard. Para asegurar la persistencia de los datos Node-RED recibe la información desde el Gateway y envía los datos a InfluxDB, una base de datos optimizada para manejar grandes volúmenes de datos de series temporales. Posteriormente, estos datos pueden ser visualizados mediante Grafana permitiéndonos crear paneles interactivos y atractivos que facilitan la interpretación de los datos almacenados en InfluxDB. A continuación, se muestra el panel construido para el proyecto, donde se muestra en un mapa las posiciones periódicas y alertas, como así también la información del dispositivo.



Pruebas y ensayos

Tiempo que tarda en obtener señal el GPS

Se desarrolló un código para que al apretar un botón el micro encienda el GPS y mida el tiempo que tardó en tomar señal. Utilizando dicho código en un día completamente despejado se obtuvieron los siguientes resultados.



```
COM3 - Tera Term VT
File Edit Setup Control Window Help
Last reset cause: External Reset
=====
EESAMR34
=====
EUID : DDADFACEDFAF5522
EUID : 0xDD 0xAD 0xFA 0xCE 0xDE 0xAF 0x55 0x22
Initialization Done

Encendiendo el GPS
Buscando señal de GPS
Time: 123331.000, Lat: -34.942955, Long: -57.937881
Tiempo: 2:38

Encendiendo el GPS
Buscando señal de GPS
Time: 123421.000, Lat: -34.942955, Long: -57.937855
Tiempo: 0:26

Encendiendo el GPS
Buscando señal de GPS
Time: 123503.000, Lat: -34.942924, Long: -57.937851
Tiempo: 0:27

Encendiendo el GPS
Buscando señal de GPS
Time: 123545.000, Lat: -34.942924, Long: -57.937855
Tiempo: 0:25

Encendiendo el GPS
Buscando señal de GPS
Time: 123633.000, Lat: -34.942902, Long: -57.937836
Tiempo: 0:25
```

Como se puede ver en la imagen, la primera vez tarda por encima de los dos minutos en tomar señal. Eso es porque se acaba de encender. Para las siguientes tomas se puede apreciar que baja notablemente el tiempo debido a que el GPS queda en modo “backup”, el cual fue explicado en la sección del GPS.

Consumo eléctrico

Se estimó el consumo eléctrico en distintas situaciones obteniendo los siguientes resultados.

Consumos	
Estado	Corriente
Sleep	2.4mA
Tomando señal de GPS	25mA
Enviando mensaje	100mA

Para este prototipo inicial se escogió una batería de LiPo de 3.7V y 2000mAh.

Suponiendo que se quiere enviar la posición cada 6 horas. El dispositivo estará en modo sleep durante 6 horas, luego aproximadamente 30 segundos tomando señal de GPS y finalmente 1

segundo enviando el mensaje, para luego volver a repetir el ciclo. Con estos datos podemos calcular el consumo por ciclo y estimar la duración en días.

$$\text{Consumo por ciclo} = \frac{2.5mA \cdot 6h + 25mA \cdot 0.00833h + 100mA \cdot 0.00027778h}{\text{Ciclo}} = \frac{15.23578 \text{ mAh}}{\text{Ciclo}}$$

$$\text{Ciclos posibles} = \frac{2000mAh}{15.23578mAh/Ciclo} \approx 131.8 \text{ Ciclos}$$

$$\text{Duración total} = 131.8 \text{ ciclos} \times \frac{6 \text{ horas}}{\text{ciclos}} = 790.8 \text{ horas} \approx 32.9 \text{ días}$$

Cabe aclarar que esto es una estimación y los tiempos en los distintos modos pueden variar, como así también no se contemplan otros escenarios como el envío de una alerta o configuración remota. Tampoco se consideran factores como la eficiencia de la batería, el envejecimiento de esta y posibles pérdidas en el circuito. Por lo que la duración real puede ser un poco menor a la estimada.

Recepción de datos en chirpstack y configuración remota

Para ver el correcto funcionamiento del dispositivo, chirpstack nos permite ver los datos recibidos mediante LoRa. También nos permite enviar datos al nodo. A continuación, se muestra la interfaz de chirpstack donde se ven los mensajes enviados y recibidos.

The screenshot displays the ChirpStack web interface for a device named 'Device_Gabriel'. The 'DEVICE DATA' tab is active, showing a list of messages. A blue box highlights a message at 7:53:59 PM with status 'up' and the text 'Envía posición cada 3 minutos'. A green box highlights a message at 7:43:23 PM with status 'up' and the text 'El nodo envía la confirmación de la nueva configuración' and 'Se configura de manera remota que el periodo sea de 3 minutos'. A red box highlights a message at 7:43:21 PM with status 'up' and the text 'Manda posición periódica cada 1 minuto'.

Below the message list, a detailed view of a message is shown. The message is decoded from Base64 format. The decoded data is as follows:

```

applicationID: "8"
applicationName: "vacas"
deviceName: "Device_Gabriel"
devEUI: "d0adfacea5522"
rxinfo: [ 1 item
txinfo: [ 3 keys
adr: true
dr: 3
fCnt: 6
fPort: 3
data: "MDAwMzQ="
objectJSON: ""
tags: [ 0 keys
confirmedUpLink: false
devAddr: "00317be6"
  
```

The decoded data is shown in a text area with the label 'MDAwMzQ='. A red arrow points from the 'data' field in the decoded data to the text area. Below the text area, there is a 'Source character set' dropdown menu set to 'UTF-8'. A 'Live mode' toggle is set to 'OFF'. A 'DECODE' button is visible. The decoded data is shown as '00034' in a text area. A red arrow points from the '00034' text to the decoded data '00(horas)03(minutos)4(sensibilidad acelerómetro)'.

7:53:59 PMup

applicationID: "B"

applicationName: "vacas"

deviceName: "Device_Gabriel"

devEUI: "ddadfaceaf5522"

rxInfo: 1 item

txInfo: 3 keys

adr: true

dr: 0

fCnt: 9

fPort: 1

data: "MjI1NDAwLjAwMC0tMzQuOTQyODQ4LS01Ny45Mzc0MTI1Mg=="

objectJSON: ""

tags: 0 keys

confirmedUplink: false

devAddr: "00317be6"

Decode from Base64 format

Simply enter your data then push the decode button.

MjI1NDAwLjAwMC0tMzQuOTQyODQ4LS01Ny45Mzc0MTI1Mg==

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page

UTF-8Source character set.

Decode each line separately (useful for when you have multiple entries).

Live mode OFFDecodes in real-time as you type or paste (supports only the UTF-8 character set).

DECODEDecodes your data into the area below.

225400 000-34 942848-57 937412-2

Nivel de batería

Hora GPS UTC

Latitud

Longitud

Resultados

Como resultado se obtuvo un prototipo bastante acorde a las expectativas iniciales. Se intento cumplir especialmente la consigna del bajo consumo, dejando de lado algunas funcionalidades pensadas al principio, como la detección de los movimientos del animal para determinar que está haciendo. Quedo como base un dispositivo capaz de enviar información sobre la posición con una alarma de aceleraciones bruscas que puede indicar una eventual caída del animal o un robo del dispositivo. Se podrían agregar más rutinas y más alarmas sobre esta base usando el sistema de mensajes diseñado.

Conclusiones

El autor de este informe opina que la ejecución del proyecto fue satisfactoria, habiéndose implementado la mayoría de las funcionalidades inicialmente propuestas y dando lugar a futuras mejoras en las que se considere una mayor utilización de los acelerómetros para brindar más información sobre el animal y tener más tipos de alarmas. Si bien el proceso de Software no siguió una metodología ni marco de trabajo en particular, se logró una buena distribución de tareas con poca superposición. Se logró separar de forma productiva el desarrollo de las distintas fases y luego se unieron sin mayores complicaciones.

Por último, este proyecto dio la oportunidad al autor de participar en el desarrollo de un sistema más complejo de lo que habría realizado por su cuenta como proyecto personal, a su vez permitiendo al autor analizar un sistema informático desde las abstracciones más altas hasta las más cercanas al hardware, teniendo que recabar información relevante de datasheets

y manuales de uso, algo que es inherente a la carrera de Ingeniería en Computación. Además, el proyecto permitió al autor mejorar sus habilidades de trabajo grupal y de comunicación entre los miembros del equipo.